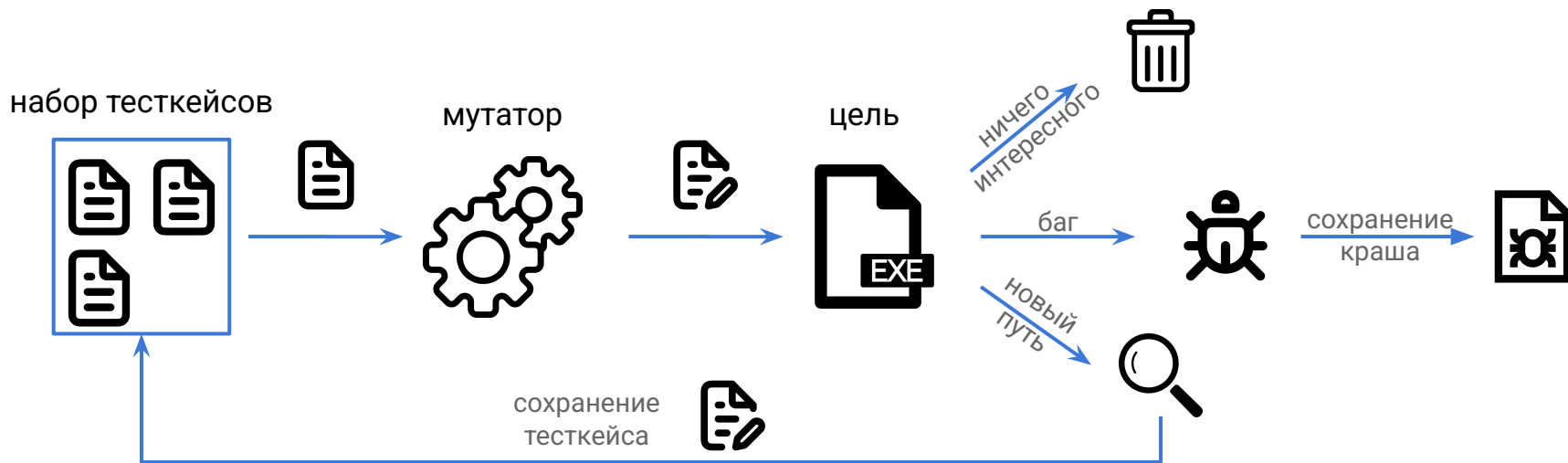


Go fuzzing workshop



Структурная схема серого фаззера



Важность фаззинга managed языков

Фаззинг способен находить не только memory corruption в C/C++. Применяя подход фаззинга для управляемых языков можно обнаружить следующие проблемы:

- Зависания и таймауты
- Race Conditions
- Undefined Behavior
- Uncaught Exceptions
- Excessive Resource Consumption
- Логические ошибки
- Memory Leaks
- ...

BDU:2024-04627: Уязвимость компонента парсера LDAP URL программного обеспечения Apache Directory LDAP API, позволяющая злоумышленнику вызвать отказ в обслуживании

Описание уязвимости Уязвимость компонента парсера LDAP URL программного обеспечения Apache Directory LDAP API связана с отсутствием контроля вводимых пользователем данных. Эксплуатация уязвимости может позволить нарушителю, действующему удалённо, вызвать отказ в обслуживании

Вендор  Apache Software Foundation

Наименование ПО  Apache Directory LDAP API

Цель для фаззинга

<https://github.com/tidwall/gjson>

 **gjson** Public

Watch 167 Fork 863 Star 14.7k

master 1 Branch 67 Tags

Go to file Add file Code

tidwall Add repo url ✓ 4d23028 · 4 months ago 315 Commits		
.github	dark mode logo	9 months ago
LICENSE	first commit	9 years ago
README.md	Update README.md	4 months ago
SYNTAX.md	chore: fix some typos in comments	10 months ago
gjson.go	Add repo url	4 months ago
gjson_test.go	Add flag for disabling HTML escaping	7 months ago

About

Get JSON values quickly - JSON parser for Go

golang json json-parser

Readme MIT license Activity 14.7k stars 167 watching 863 forks Report repository

Подготовка окружения

```
1 FROM golang:1.24
2
3 WORKDIR /go/src
4
5 # get gjson source code
6 RUN wget https://github.com/tidwall/gjson/archive/refs/tags/v1.18.0.tar.gz
7 RUN tar xf v1.18.0.tar.gz && rm v1.18.0.tar.gz
```

Собираем docker образ:

```
#bash
$ git clone https://github.com/lenix123/go_fuzz_workshop.git
$ cd go_fuzz_workshop
$ docker build --tag=gjson_workshop_img .
```

Запускаем контейнер:

```
#bash
$ docker run -it -v "$(pwd)/artifacts:/home/fuzz/artifacts:ro" --name=gjson_fuzz gjson_workshop_img
```

Фаззинг в Go с помощью встроенного фаззера

Go первый язык программирования, который добавил нативную поддержку фаззинга. С версии 1.18 Go поставляется с [поддержкой фаззинга](#). Это делает написание фаззинг обёрток схожим с написанием unit-тестов. При этом фаззинг обёртка может принимать сразу несколько параметров простых типов (`string`, `[]byte`, `int`, `uint...`), что также упрощает работу с целевой функцией ([tutorial](#)).

Применяем патч:

```
#bash
$ cd /go/src/gjson-1.18.0
$ git apply /home/fuzz/artifacts/gjson.patch
```

```
4  +++ b/gjson_test.go
5  @@ -15,6 +15,12 @@ import (
6      "github.com/tidwall/pretty"
7  )
8
9  +func FuzzParseJSON(f *testing.F) {
10 +    f.Fuzz(func(t *testing.T, orig string) {
11 +        Parse(orig)
12 +    })
13 +}
```

Запуск фаззинга

Подготавливаем входной корпус:

```
#bash
$ mkdir -p testdata/fuzz/FuzzParseJSON
$ cp /home/fuzz/artifacts/corpus/* testdata/fuzz/FuzzParseJSON/
```

Запускаем фаззинг FuzzParseJSON:

```
#bash
$ go test -fuzz=Fuzz -run=FuzzParseJSON -parallel=1
```

Наработанный корпус хранится в ``$GOPATH/fuzz``. Но это значение можно изменить при запуске фаззинга с помощью флага `-test.fuzzcachedir`

Сбор покрытия

Переносим наработанный корпус в рабочую директорию:

```
#bash
$ cp $( go env GOCACHE )/fuzz/$( go list )/FuzzParseJSON/* testdata/fuzz/FuzzParseJSON
```

Прогоняем тест с наработанным корпусом:

```
#bash
$ go test -coverprofile=coverage.out -run=FuzzParseJSON -v
```

Генерируем html отчёт о покрытии:

```
#bash
$ go tool cover -html=coverage.out -o ./coverage.html
```

Визуализация покрытия

Переносим сгенерированный html отчёт на хост:

```
#bash на хосте  
$ docker cp gjson_fuzz:/go/src/gjson-1.18.0/coverage.html .
```

Открываем html отчёт с помощью браузера:

```
func Parse(json string) Result {  
    var value Result  
    i := 0  
    for ; i < len(json); i++ {  
        if json[i] == '{' || json[i] == '[' {  
            value.Type = JSON  
            value.Raw = json[i:] // just take the entire raw  
            break  
        }  
        if json[i] <= ' ' {  
            continue  
        }  
        switch json[i] {  
            case '+', '-', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9',  
                'i', 'I', 'N':  
                value.Type = Number  
                value.Raw, value.Num = tonum(json[i:])  
            }  
        }  
    }  
}
```